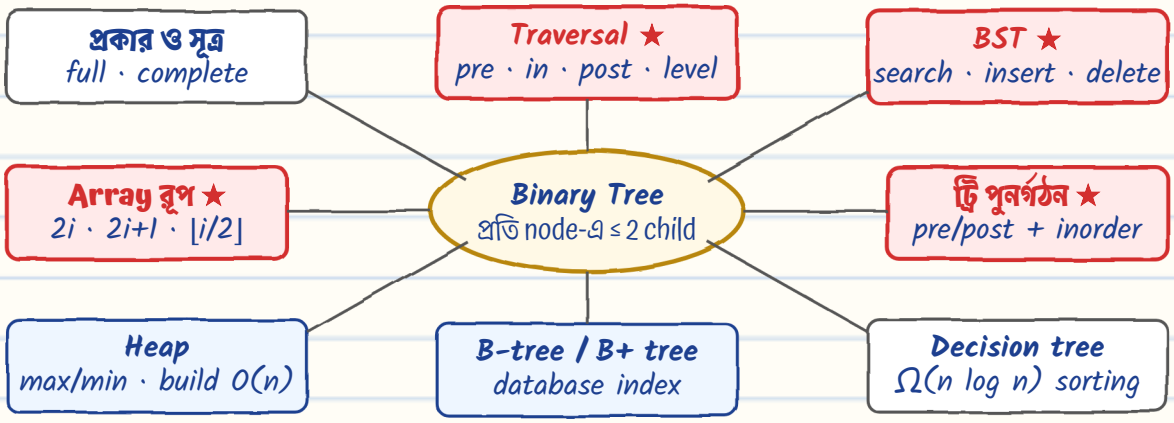
ট্রি ওয়ার্কশপ: Build, Traverse, BST ও Heap

Traversal · BST · array রূপ · ট্রি পুনর্গঠন · heap · B-tree: ১১ শিটে পরীক্ষার প্রস্তুতি



সংকেত হলুদ=মূল শব্দ · ● =অবশ্যই মুখস্থ · ▢ =স্মৃতি-কৌশল · ▢ =পরীক্ষার ফাঁদ · লাল node = আলোচ্য node

সূচিপত্র

1. পরিভাষা, প্রকারভেদ ও node সূত্র ... পৃ.২
2. Traversal ও expression tree ... পৃ.৩
3. BST: search ও insert ... পৃ.৪
4. BST: delete (৩টি case) ... পৃ.৫
5. Array representation ... পৃ.৬
6. ট্রি পুনর্গঠন ১: inorder সহ ... পৃ.৭
7. ট্রি পুনর্গঠন ২: pre + post ... পৃ.৮
8. এক শিটে heap ও B-tree ... পৃ.৯
9. Decision tree lower bound + রাতের আগের শিট ... পৃ.১০
10. অনুশীলন MCQ + লিখিত উত্তর ... পৃ.১১

সিলেবাসের যে অংশ "... **trees** ...; Advanced: **heaps**, Fibonacci heaps, **B-trees**।" Heap/B-tree-এর গভীর অংশ: Heaps ও B-Trees লেসন।

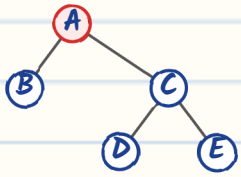
High-yield তালিকা

① BST-এর inorder = sorted
② Postorder দিয়ে tree মোছা
③ 0-indexed left = 2i+1
④ সর্বোচ্চ node = 2^(h+1)−1
⑤ Full tree: L = I + I
⑥ ২ child → successor
⑦ Sorting ≥ Ω(n log n)

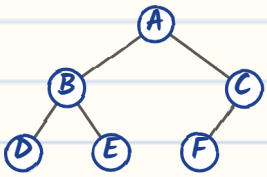
১ পরিভাষা, Binary Tree-এর প্রকারভেদ ও Node সূত্র ★ MCQ সূত্র

Tree একটি *hierarchical, non-linear* structure: একটি **root**, কোনো cycle নেই; n node-এ **n − 1 edge** (file system, HTML DOM)। *Binary tree*: প্রতিটি node-এর **সর্বোচ্চ ২টি** child, *left* ও *right* (একা left child ≠ একা right child)।

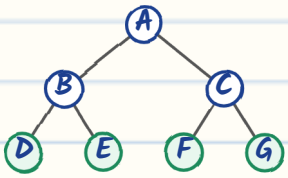
শব্দ	অর্থ	শব্দ	অর্থ
Root	parent নেই এমন একমাত্র node	Leaf	কোনো child নেই (external node)
Internal node	≥ ১টি child আছে	Degree	child-এর সংখ্যা
Depth / level	root থেকে edge সংখ্যা (root = 0)	Height	দীর্ঘতম root-to-leaf path-এর edge



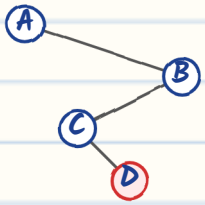
Full / proper
0 বা 2টি child



Complete
শেষ level বাম থেকে পূর্ণ



Perfect
সব leaf এক level-এ



Skewed
প্রতিটিতে 1 child

রাশি (height h = edge গণনা, root level 0)	সূত্র	যাচাই
Height h-এর tree-তে সর্বোচ্চ node (level i-তে ≤ 2^i)	$2^{(h+1)} - 1$	h = 3 → 15
N level-এ সর্বোচ্চ node (root = level 1)	$2^N - 1$	N = 4 → 15
Height h-এর tree-তে সর্বনিম্ন node	h + 1 (skewed)	h = 3 → 4
n node-এ সর্বনিম্ন height = complete tree-এর height	$\lfloor \log_2 n \rfloor$	n = 10 → 3
Leaf বনাম ২-child node (যেকোনো binary tree)	$n_0 = n_2 + 1$	perfect h=2: 4 = 3 + 1
I internal node-এর full tree	L = I + 1, n = 2I + 1	I = 10 → L = 11
n node-এ ভিন্ন আকৃতি (Catalan)	(2n)! / ((n+1)! n!)	n = 3 → 5

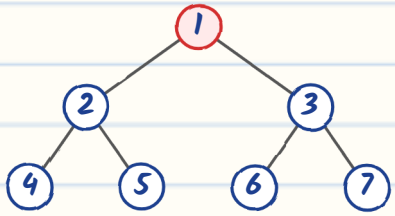
প্রমাণ (লিখিত প্রিয়) **Full (proper) binary tree-এ leaf = internal node + ১।** ধরি I = internal node, L = leaf। Edge দুইভাবে গুনি: root ছাড়া প্রতিটি node-এর একটি parent edge, তাই edge = I + L − 1; প্রতিটি internal node-এর ২টি child, তাই edge = 2I। অতএব 2I = I + L − 1, অর্থাৎ $L = I + 1$। ■

পরীক্ষার ফাঁদ **Height-এর রীতি।** এখানে height = edge গণনা (একা root = 0)। কিছু বই **node** গোনে (একা root = 1), তখন "N level" = $2^N - 1$। রীতিটি লিখে দিন।

স্মৃতি-কৌশল প্রতি level দ্বিগুণ: 1, 2, 4, 8 ..., তাই h+1টি পূর্ণ level = "পরের 2-এর ঘাত বিয়োগ ১"। Perfect ⊂ complete ও perfect ⊂ full, কিন্তু full ⇏ complete।

২ Traversal: Pre, In, Post ও Level Order ★ সর্বাধিক পরীক্ষিত

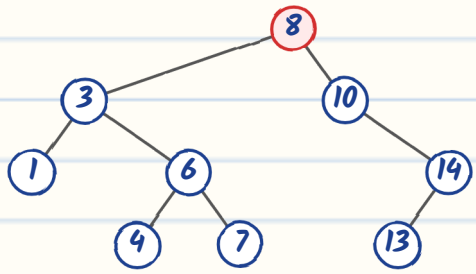
Traversal প্রতিটি node একবার visit। ৩টি **depth-first** order-এর পার্থক্য শুধু root কখন visit হয়; **level order** breadth-first (queue)। সবই O(n)।



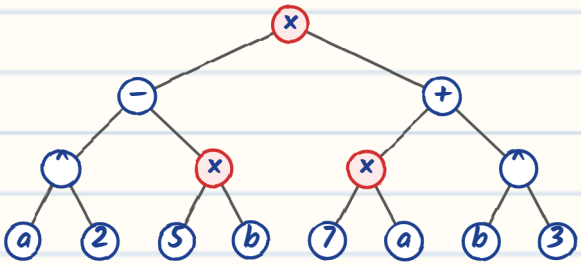
Preorder (Root L R): 1 2 4 5 3 6 7
Inorder (L Root R): 4 2 5 1 6 3 7
Postorder (L R Root): 4 5 2 6 7 3 1
Level order (queue): 1 2 3 4 5 6 7
root 1: pre-তে প্রথমে, in-এ মাঝে, post-এ শেষে

Traversal	ক্রম	সাধারণ ব্যবহার
Preorder	Root L R	tree copy / serialise, prefix (Polish) রূপ
Inorder	L Root R	BST-এর key **ascending** ক্রমে, infix রূপ
Postorder	L R Root	tree **delete/free**, postfix মূল্যায়ন, folder size
Level order	level ধরে (queue)	BFS, level-ভিত্তিক print, heap array ক্রম

উদাহরণ খালি BST-এ **8 3 10 1 6 14 4 7 13** insert করে সব ক্রমে traverse করুন, অস্বাভাবিক **Right → Left → Root** সহ।



Pre: **8 3 1 6 4 7 10 14 13**
In: 1 3 4 6 7 8 10 13 14 (sorted ✓)
Post: **1 4 7 6 3 13 14 10 8**
Level: **8 3 10 1 6 14 4 7 13**
R → L → Root: **13 14 10 7 4 6 1 3 8** (= উল্টো preorder)



$X = (a^2 - 5b) \times (7a + b^3)$-এর **Expression tree**: operator ভেতরে, operand leaf-এ।

Prefix (pre): × − ^ a 2 × 5 b + × 7 a ^ b 3
Postfix (post): a 2 ^ 5 b × − 7 a × b 3 ^ + ×

স্মৃতি-কৌশল **Pre / In / Post = Root কোথায় বসে**। Tree-র বাইরের রেখা ধরে ঘুরুন; node-এর **বাম** পাশে (pre), **নিচে** (in) বা **ডান** পাশে (post) লিখুন।

পরীক্ষার ফাঁদ Parent-এর আগে child delete → postorder। Inorder sorted হয় **শুধু BST-তে**।

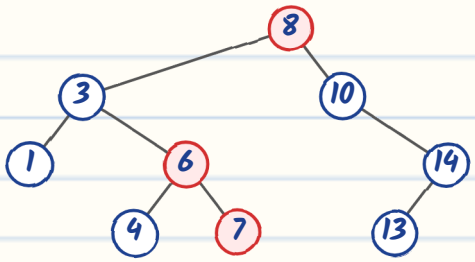
৩ Binary Search Tree: Search ও Insert ★ Bank ও PSU লিখিত

BST property **প্রতিটি** node-এর জন্য: left subtree-এর সব key ছোট, right subtree-এর সব key বড় (duplicate নেই, যদি না "সমান হলে ডানে" জাতীয় নিয়ম বলা হয়)। ফল: **inorder = sorted**।

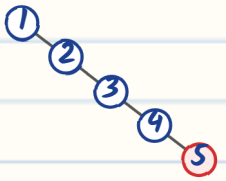
```
/* search: iterative */
node* search(node* t, int k) {
  while (t && t->key != k)
    t = (k < t->key) ? t->left
                     : t->right;
  return t;   /* NULL = absent */
}
```

```
/* insert: recursive */
node* insert(node* t, int k) {
  if (!t) return newNode(k);
  if (k < t->key)
    t->left = insert(t->left, k);
  else if (k > t->key)
    t->right = insert(t->right, k);
  return t;   /* duplicate ignored */
}
```

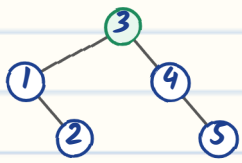
উদাহরণ খালি BST-এ **8 3 10 1 6 14 4 7 13** insert: প্রতিটি key root থেকে নামে, ছোট হলে বামে, বড় হলে ডানে, যতক্ষণ না খালি link পায়।



4: 4 < 8, 4 > 3, 4 < 6 → left of 6
13: 13 > 8, 13 > 10, 13 < 14 → left of 14
search 7: 8, 3, 6, 7 → পাওয়া গেল, 4 তুলনা
search 5: 8, 3, 6, 4, then NULL
→ 4 তুলনার পর পাওয়া যায়নি;
5 ঠিক ওখানেই insert হবে (4-এর ডানে)



keys 1 2 3 4 5: h = 4



keys 3 1 4 2 5: h = 2

প্রতিটি operation **O(h)**
balanced: $h \approx \log_2 n \rightarrow O(\log n)$
skewed: $h = n - 1 \rightarrow O(n)$
সমাধান: AVL / red-black tree

স্মৃতি-কৌশল **"ছোট বামে, বড় ডানে, NULL-এ থামো।"** **প্রথম** key সবসময় root; sorted input-এ BST linked list হয়ে যায়।

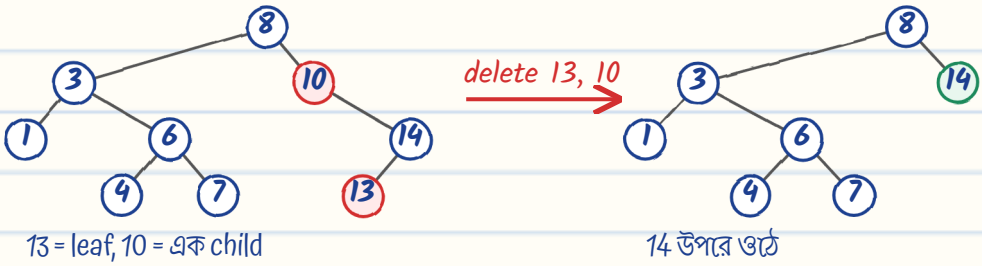
পরীক্ষার ফাঁদ BST নিজে থেকে balanced **নয়**: search O(log n) শুধু balanced হলে, worst case-এ O(n)। Min = সবচেয়ে বামের node, max = সবচেয়ে ডানের।

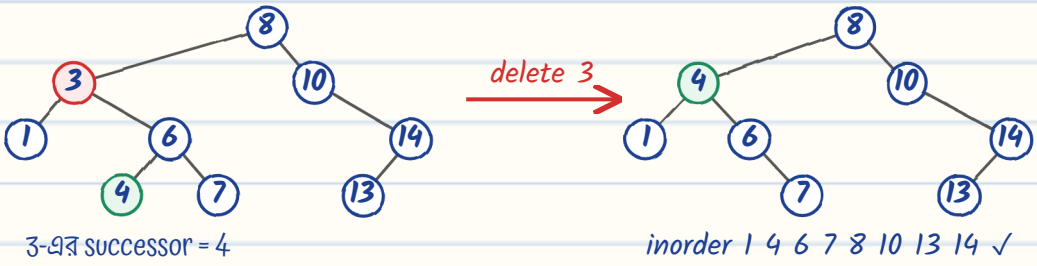
৪ BST Delete: তিনটি Case ★ লিখিত প্রিয়

১ **Leaf**: সরাসরি সরান। *13 delete → 14-এর left child থাকে না।*

২ **একটি child**: *child-কে parent-এর সাথে যুক্ত করুন। 10 delete → 14 উপরে ওঠে।*

৩ **দুটি child**: *inorder successor (right subtree-এর সবচেয়ে ছোট key)* কপি করুন, তারপর সেই *successor delete* করুন (তার ≤ ১টি child)। **Predecessor** (বামের সবচেয়ে বড়) দিয়েও চলে।





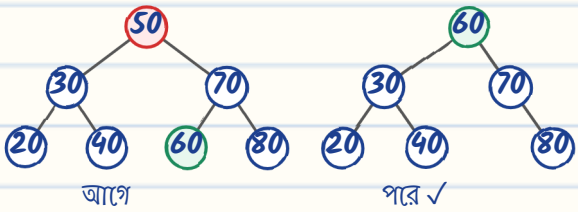
উদাহরণ ১ **Root 8** *delete: successor =* **10** *(left child নেই), তাই 10 উপরে ওঠে ও 14 তার জায়গা নেয় → 10(3(1, 6(4, 7)), 14(13))। Predecessor নিলে* **7** *root হয়।*

উদাহরণ ২: পূর্ণ লিখিত প্রশ্ন *(ক) 50, 30, 70, 20, 40, 60, 80 insert। (খ) Postorder লিখুন। (গ) 50 delete করে আবার আঁকুন।*

(ক) 50 root; 30, 70; তারপর 20, 40 ও 60, 80।

(খ) Postorder: 20 40 30 60 80 70 50

(গ) Successor = **60**: *উপরে কপি, leaf সরান।*



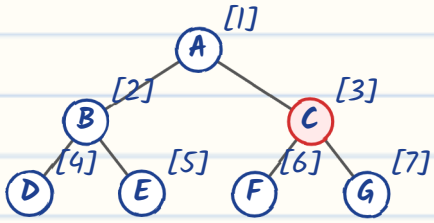
স্মৃতি-কৌশল *Successor =* **"একবার ডানে, তারপর পুরো বামে"**; *predecessor =* **"একবার বামে, তারপর পুরো ডানে"**। *শেষে inorder sorted কিনা দেখুন।*

পরীক্ষার ফাঁদ *Case ৩-এ successor-এর* **key** *কপি হয়; "right child দিয়ে বদলাও" ভুল, যখন সেই child-এর left subtree আছে (3 delete: 6-এর left child 4)।*

৫ Binary Tree-কে Array-তে রাখা ★ লিখিত + BCS MCQ

Sequential representation Root index 1-এ, তারপর level order-এ বাম থেকে ডানে position নম্বর। অনুপস্থিত node-এর ঘর খালি থাকে; pointer-এর কাজ করে index-এর হিসাব।

	Root	i-এর left child	i-এর right child	i-এর parent
1-indexed	1	2i	2i + 1	⌊i / 2⌋
0-indexed (C, Java)	0	2i + 1	2i + 2	⌊(i − 1) / 2⌋



0	1	2	3	4	5	6	7
–	A	B	C	D	E	F	G

C at i = 3: left = 2·3 = 6 → F, right = 7 → G
parent of C = ⌊3/2⌋ = 1 → A

উদাহরণ: ফাঁকসহ tree শিট ৪-এর BST: 14 = 10-এর right → 2·3+1 = 7; 13 = 14-এর left → 2·7 = 14।

1	2	3	4	5	6	7	8	9	10	11	12	13	14
8	3	10	1	6	–	14	–	–	4	7	–	–	13

৯টি key-তে **১৪টি ঘর**; n node-এর right-skewed tree-তে লাগে $2^n - 1$।

উদাহরণ: postorder → array Node a…i, postorder **a b f c h d e g i**। Tree-কে **complete** ধরুন (একটি traversal একা অস্পষ্ট): তার position-গুলোর postorder **8 9 4 5 2 6 7 3 1**, তাই 8 = a, 9 = b, 4 = f, …

1	2	3	4	5	6	7	8	9
i	h	g	f	c	d	e	a	b

tree i(h(f(a, b), c), g(d, e))
postorder a b f c h d e g i ✓

Array বনাম linked Array: parent O(1)-এ, complete tree (heap)-এর জন্য আদর্শ, ফাঁকা tree-তে অপচয়। Linked: n + 1টি NULL pointer, যেকোনো আকৃতি।

স্মৃতি-কৌশল **দ্বিগুণ** করলে নিচে-বামে, **অর্ধেক** করলে উপরে (1-indexed)।

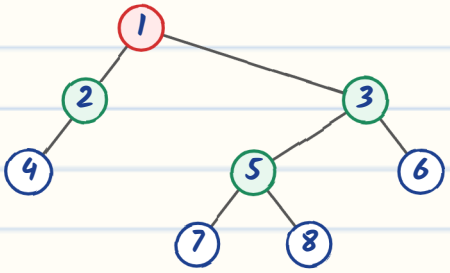
পরীক্ষার ফাঁদ Indexing পড়ুন! "Zero-based, i-এর left child?" → 2i + 1, 2i নয়। Complete tree-এর leaf (1-indexed): ⌊n/2⌋+1 … n।

৬ Inorder + আরেকটি থেকে Tree পুনর্গঠন ★ BUET-ধাঁচের লিখিত

নিয়ম **Preorder**-এ root **প্রথমে**; **postorder**-এ **শেষে**। Inorder বাকিটাকে left ও right subtree-তে ভাগ করে। Recursion চালান। **Inorder + pre বা post** = একটিমাত্র tree।

উদাহরণ ১: preorder + inorder

Inorder: **4 2 1 7 5 8 3 6** · Preorder: **1 2 4 3 5 7 8 6**



১. Pre[0] = **1** = root। Inorder: বামে {4 2}, ডানে {7 5 8 3 6}।

২. বাম: root **2**; inorder "4 2" → 4 তার **left**।

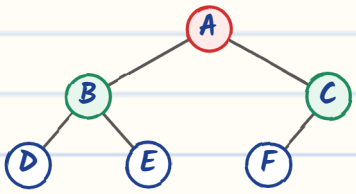
৩. ডান, pre 3 5 7 8 6: root **3**; inorder "7 5 8 | 3 | 6"।

৪. {7 5 8}, pre 5 7 8: root **5**, left 7, right 8; 6 হলো 3-এর right।

Pre ✓ ও in ✓ মিলিয়ে নিন; postorder = **4 2 7 8 5 6 3 1**।

উদাহরণ ২: postorder + inorder

Inorder: **D B E A F C** · Postorder: **D E B F C A**।



১. Root = শেষটি = **A**; বামে {D B E}, ডানে {F C}।

২. বামের post "D E B" → root **B**, D left, E right।

৩. Post "F C" → root **C**; inorder "F C" → F তার **left**।

ফলাফলের preorder: A B D E C F।

উদাহরণ ৩: ছাপার ভুলসহ ক্লাসিক

Preorder **a b d e c f g**, কিন্তু inorder মাত্র ৫ অক্ষরের "d e a f g": inorder-এ ৭টি key-ই থাকতে হবে। **d b e a f c g** ধরলে tree a(b(d, e), c(f, g)), postorder **d e b f g c a**। ধরে নেওয়াটা লিখে দিন।

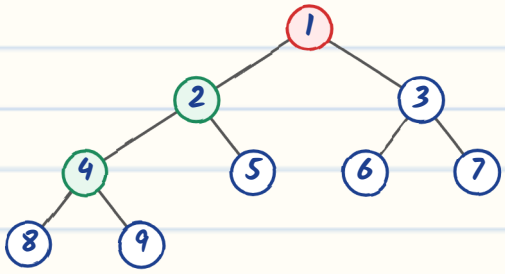
স্মৃতি-কৌশল **"Pre/Post দেখায়, In ভাগ করে।"** Pre/post বলে কে root; inorder বলে কে কোথায় যাবে।

৭ Preorder + Postorder, এবং BST শর্টকাট ★ Bank ও BUET

পদ্ধতি (শুধু full tree) Root = pre[0] = post[শেষ]। Preorder-এর **পরের** key হলো **left** subtree-এর root; postorder-এ সেটি পর্যন্ত সব left subtree, বাকিটা right। Recursion চালান।

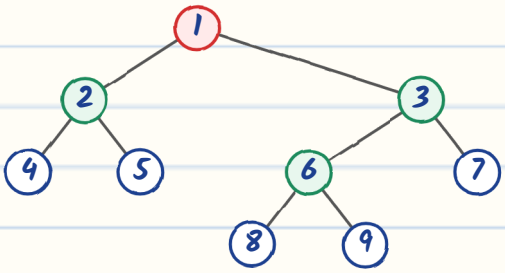
উদাহরণ ১ Pre: **1 2 4 8 9 5 3 6 7** · Post: **8 9 4 5 2 6 7 3 1**

১. Root 1, পরের key **2**: post-এ 2 পর্যন্ত → বামে {8 9 4 5 2}, ডানে {6 7 3}।
২. Root 2, পরের key **4**: post-এ 4 পর্যন্ত → {8 9 4}; 5 হলো 2-এর right।
৩. 4(8, 9); ডান অংশ 3(6, 7)।



উদাহরণ ২ Pre: **1 2 4 5 3 6 8 9 7** · Post: **4 5 2 8 9 6 7 3 1**

১. Root 1, পরের key 2: post-এ 2 পর্যন্ত = {4 5 2} → 2(4, 5)।
২. ডান: pre 3 6 8 9 7 → root 3, পরের key 6 → 6(8, 9); 7 হলো 3-এর right।



পরীক্ষার ফাঁদ কোনো node-এর একটিমাত্র child থাকলে **Pre + post** অস্পষ্ট: pre "A B", post "B A"-তে B left **বা** right দুটোই হতে পারে। Inorder + যেকোনো একটি সবসময় অনন্য; একটি traversal একা কখনো নয়।

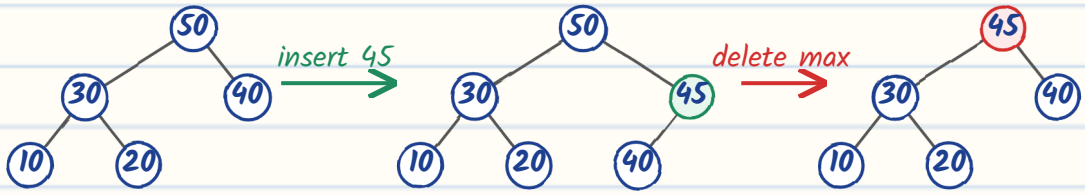
BST শর্টকাট BST-এ inorder মানে sorted key, তাই শুধু preorder-ই যথেষ্ট: pre 8 3 1 6 4 7 10 14 13 → sort → পুনর্গঠন (বা ওই ক্রমে insert) → শিট ৪-এর BST।

অনুশীলন Pre 1 5 8 7 4 9 3 · In 8 5 7 1 9 4 3 → 1(5(8, 7), 4(9, 3)): সাধারণ binary tree, BST নয়।

৮ Heap ও B-tree: পরীক্ষার মূল কথা

★ Priority queue · DB index

Heap Array-তে রাখা একটি complete binary tree: **max-heap**-এ parent ≥ children (root = max), **min-heap**-এ parent ≤ children। Sibling-দের মধ্যে ক্রম নেই।

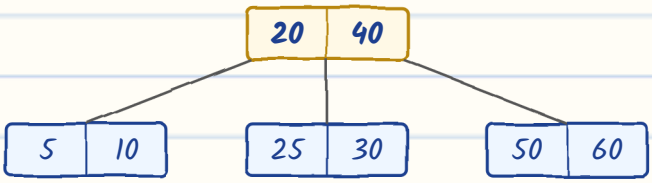


১. **45 insert** index 6-এ; parent ⌊6/2⌋ = 3-এ 40 < 45 → swap; parent 50 ≥ 45 → থামুন (**sift-up**, O(log n))।

২. **Delete max**: 50 সরান, শেষ key 40 root-এ আনুন; child 45 > 40 → swap (**sift-down**, O(log n))।

৩. **Build-heap**: node ⌊n/2⌋ থেকে 1 পর্যন্ত sift-down: মোট **O(n)**, O(n log n) নয়। Find-max O(1); heapsort O(n log n)।

Order m-এর B-tree একটি self-balancing multi-way search tree: প্রতি node-এ ≤ **m** child ও ≤ **m − 1** key; non-root internal node-এ ≥ **⌈m/2⌉** child; সব leaf **একই level-এ**। Height = $O(\log_m n)$।



Order 3: ≤ 3 children, ≤ 2 keys
keys < 20 | 20...40 | > 40
node = একটি disk block
সব leaf এক level-এ ✓
পূর্ণ node → split, মাঝেরটা উপরে

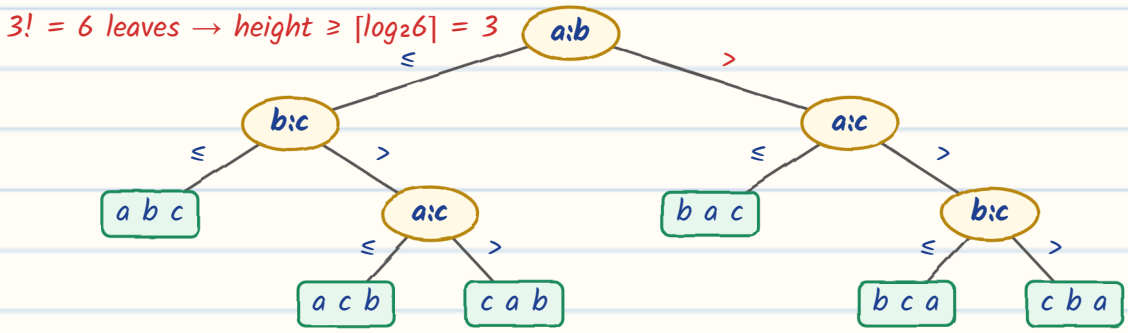
Database কেন B-tree নেয় Node = শত শত key-সহ একটি disk block: লক্ষ লক্ষ row মাত্র ৩–৪ level-এ = ৩–৪ বার read। **B+ tree**: record শুধু linked leaf-এ, তাই range query মানে leaf scan (MySQL InnoDB, PostgreSQL)।

পরীক্ষার ফাঁদ **Heap BST নয়**: heap-এ search O(n)। "Order m" = সর্বোচ্চ m child (Knuth); CLRS-এ minimum degree t। কোনটি নিচ্ছেন বলুন।

স্মৃতি-কৌশল **Heap = priority, BST = search, B-tree = disk।** Insert-এ sift **up**, delete-এ sift **down**।

৯ Decision Tree: Sorting-এ কেন Ω(n log n) লাগে ★ BCS MCQ

Decision tree model n key-এর যেকোনো comparison sort একটি binary tree: internal node = একটি তুলনা ($a \le b$?), **leaf** = একটি চূড়ান্ত ক্রম, একটি run = একটি root-to-leaf path। তাই **worst-case তুলনা = height**।



১. n টি ভিন্ন key-এর **n!** ক্রম, প্রতিটির নিজস্ব leaf লাগে → leaf ≥ n!।

২. Height h-এর binary tree-তে সর্বোচ্চ **2^h** leaf → $2^h \ge n!$ → $h \ge \log_2(n!)$।

৩. $n! \ge (n/2)^{(n/2)} \Rightarrow \log_2(n!) \ge (n/2)\cdot\log_2(n/2) = \Omega(n \log n)$। Merge sort ও heapsort তাই optimal।

পরীক্ষার ফাঁদ Counting/radix sort comparison sort **নয়**: $O(n + k)$ সম্ভব। ML-এ "decision tree" একটি **classifier**।

সংখ্যা $\lceil \log_2 n! \rceil$: $n = 3 \to 3$, $n = 4 \to 5$, $n = 5 \to 7$। Sorted array-তে search: $\Omega(\log n)$।

রাতের আগের শিট

- n node → n − 1 edge, n + 1 NULL link
- height h-এ সর্বোচ্চ node = $2^{(h+1)} - 1$; N level = $2^N - 1$
- full tree: $L = I + 1$; যেকোনো tree: $n_0 = n_2 + 1$
- complete tree-এর height = $\lfloor \log_2 n \rfloor$
- Pre = Root L R · In = L Root R · Post = L R Root
- BST inorder = sorted; postorder দিয়ে tree মোছা
- level order-এ queue; DFS-এ stack
- R-L-Root = উল্টো preorder
- BST ops O(h): balanced O(log n), skewed O(n)
- delete, ২ child → inorder successor
- 1-indexed: 2i, 2i+1, ⌊i/2⌋; 0-indexed: 2i+1, 2i+2
- inorder + pre/post → অনন্য tree
- heap: insert/delete O(log n), build O(n)
- B-tree: সব leaf এক level-এ, node = disk block
- sorting lower bound: $h \ge \log_2 n! = \Omega(n \log n)$
- ৪টি key-তে ≥ ৫টি তুলনা

স্মৃতি-কৌশল "**n! leaf, 2^h জায়গা**": 2^h ঘরের tree-তে n!টি উত্তর ধরাতে হবে, তাই $h \ge \log_2 n!$।

✓ অনুশীলন MCQ ও একটি মডেল লিখিত উত্তর

১। Binary search tree-এর কোন traversal key-গুলো ascending ক্রমে দেয়?
ক) Preorder খ) Postorder **গ) Inorder** ঘ) Level order

২। Comparison sorting-এর lower bound প্রমাণে কোন পদ্ধতি ব্যবহৃত হয়?
ক) Decision tree খ) Recursion tree গ) Master theorem ঘ) Dynamic programming

৩। প্রতিটি child-কে তার parent-এর আগে delete করতে ___ order লাগে।
ক) Pre খ) In গ) Level **ঘ) Post**

৪। Zero-based array-তে রাখা complete binary tree-তে index i-এর left child কোথায়?
ক) 2i **খ) 2i + 1** গ) i + 1 ঘ) i − 1

৫। Height 3-এর binary tree-তে সর্বোচ্চ node কত (root-এর height 0)?
ক) 8 খ) 7 **গ) 15** ঘ) 16

৬। একটি full binary tree-তে ১০টি internal node। Leaf কয়টি?
ক) 9 খ) 10 গ) 20 **ঘ) 11**

৭। n node-এর BST-তে search-এর worst-case সময়:
ক) O(1) **খ) O(n)** গ) O(log n) ঘ) O(n log n)

৮। n টি অসাজানো key থেকে bottom-up binary heap তৈরিতে সময়:
ক) O(n) খ) O(log n) গ) O(n log n) ঘ) $O(n^2)$

৯। দুটি child-সহ BST node delete করলে তার key-এর জায়গায় বসে:
ক) তার parent খ) তার left child **গ) তার inorder successor** ঘ) root

১০। Comparison দিয়ে ৪টি key sort করতে worst case-এ ন্যূনতম কতটি তুলনা লাগে?
ক) 4 **খ) 5** গ) 6 ঘ) 24

উত্তর: ১গ ২ক ৩ঘ ৪খ ৫গ ৬ঘ ৭খ ৮ক ৯গ ১০খ

লিখিত (১০ নম্বর): একটি binary tree-এর inorder **D B H E A I F J C G** এবং preorder **A B D E H C F I J G**। Tree-টি আঁকুন ও postorder লিখুন।

মডেল উত্তর:
Root = preorder-এর প্রথম = **A**; inorder ভাগ করে: বামে {D B H E}, ডানে {I F J C G}। [২]
বাম: pre B D E H → root B; inorder "D | B | H E" → D হলো B-এর left; {H E}, pre E H → E, যার left child H। [২]
ডান: pre C F I J G → root C; inorder "I F J | C | G" → G হলো C-এর right; {I F J}, pre F I J → F(I, J)। [২]
Tree: A(B(D, E(H, –)), C(F(I, J), G))। [২] Postorder: **D H E B I J F G C A**; আঁকা tree থেকে preorder ও inorder পড়ে মিলিয়ে নিলে ঠিক ✓। [২]

পরীক্ষার আগে ✓ ☐ height-এর রীতি বলুন ☐ outline ধরে traversal ☐ BST delete-এ successor নিয়ম ☐ indexing ভিত্তি (0 না 1) দেখুন ☐ পুনর্গঠনের পর দুটি traversal মিলিয়ে নিন ☐ heap ≠ BST